DESARROLLO DE APLICACIONES CON OBJETOS · UNIDAD 1

Comprensiones, lambda, map, filter y reduce

Los ejemplos de la clase ordenados y con su salida, y el ejercicio de la clase resuelto

Cuadro de las 5 herramientas | Ejemplos que andan | Ejercicio resuelto

Cómo usar esta guía

Los ejemplos de la clase 2 (programación funcional) ordenados y con su salida, y el ejercicio de la clase resuelto. El código está en el ZIP para correrlo. Probado con Python 3.11.

Herramienta	Qué hace	Forma	Devuelve
Comprensión	Arma una lista (o diccionario, o conjunto) recorriendo otra, en una línea	`[x * 2 for x in lista if x > 0]`	la colección nueva
lambda	Una función sin nombre, de una sola expresión	`lambda x: x ** 2`	una función
map	Aplica una función a cada elemento	`map(lambda x: x ** 2, lista)`	un iterador (usar `list(...)`)
filter	Se queda con los elementos para los que la función da `True`	`filter(lambda x: x % 2 == 0, lista)`	un iterador
reduce	Acumula de a dos hasta dejar un solo valor (`from functools import reduce`)	`reduce(lambda a, b: a + b, lista)`	un valor

Qué se corrigió de los ejemplos de la cátedra

- En `comprension.py`, el texto decía “cubos” pero se calculaba `item ** 2` (cuadrados).
- En `map_filter_reduce.py` los ejemplos estaban comentados, y el de `map` que decía “cuadrados” calculaba `x % 2 == 0` (si es par). Acá están andando y cada uno hace lo que dice.
- Los ejemplos usaban la librería `rich` (hay que instalarla): acá se usa `print` común.

1 Comprensiones

La forma es `[expresión for elemento in iterable if condición]`. Con llaves y `clave: valor` se arma un diccionario.

comprensiones.py

```python
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Forma tradicional: un for que va armando la lista
pares = []
for n in numeros:
    if n % 2 == 0:
        pares.append(n)
print("Pares (con for):", pares)

# Por comprensión: [expresión for elemento in iterable if condición]
print("Pares (comprensión):", [n for n in numeros if n % 2 == 0])
print("Siguiente de cada par:", [n + 1 for n in numeros if n % 2 == 0])
print("Cuadrados de los múltiplos de 3:", [n ** 2 for n in numeros if n % 3 == 0])

# Comprensión de diccionarios: {clave: valor for ...}
personas = [
    {"nombre": "Alice", "edad": 30, "puesto": "Data Scientist"},
    {"nombre": "Bob", "edad": 35, "puesto": "Data Engineer"},
    {"nombre": "Chris", "edad": 33, "puesto": "Machine Learning Engineer"},
]
print("Puestos con 'Data':", {p["nombre"]: p["puesto"] for p in personas if "Data" in
p["puesto"]})
```

Salida al correrlo

```
Pares (con for): [2, 4, 6, 8, 10]
Pares (comprensión): [2, 4, 6, 8, 10]
Siguiente de cada par: [3, 5, 7, 9, 11]
Cuadrados de los múltiplos de 3: [9, 36, 81]
Puestos con 'Data': {'Alice': 'Data Scientist', 'Bob': 'Data Engineer'}
```

2 Funciones lambda

Sirven para pasar una función corta como argumento (a `sorted`, `map`, `filter`...). En Python las funciones son valores: se guardan en variables como cualquier otro dato.

lambdas.py

```python
# lambda argumentos: expresión  (una función sin nombre, de una sola expresión)
cuadrado = lambda x: x ** 2
print(cuadrado(3))

cuadrado_mayor_a_diez = lambda x: x ** 2 > 10
print([cuadrado_mayor_a_diez(n) for n in (2, 3, 4, 5)])

# Las funciones son valores: se pueden guardar en variables y pasar como argumento
imprimir = print
imprimir("Hola desde una variable que guarda una función")
print(sorted(["Bermuda", "Aruba", "Key Largo"], key=lambda s: len(s)))
```

Salida al correrlo

```
9
[False, False, True, True]
Hola desde una variable que guarda una función
['Aruba', 'Bermuda', 'Key Largo']
```

3 map, filter y reduce

map_filter_reduce.py

```python
from functools import reduce

lista = [1, 2, 3, 4, 5]

# map: aplica la función a cada elemento
print("Cuadrados:", list(map(lambda x: x ** 2, lista)))

# filter: se queda con los elementos para los que la función da True
numeros = list(range(-10, 11))
print("Negativos pares o positivos:", list(filter(lambda x: (x <= 0 and x % 2 == 0) or x > 0,
numeros)))

# reduce: acumula de a dos, de izquierda a derecha, hasta dejar un solo valor
print("Suma:", reduce(lambda acumulado, x: acumulado + x, lista))       # ((((1+2)+3)+4)+5)
print("Producto:", reduce(lambda acumulado, x: acumulado * x, lista))
```

Salida al correrlo

```
Cuadrados: [1, 4, 9, 16, 25]
Negativos pares o positivos: [-10, -8, -6, -4, -2, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Suma: 15
Producto: 120
```

4 Ejercicio de la clase

Enunciado

Punto de partida. Dada la siguiente lista de strings en Python, realiza las actividades propuestas utilizando comprensiones y funciones de orden superior:

```
lst = ["Aruba", "Jamaica", "Bermuda", "Bahama", "Key Largo", "Montigo"]
```

1. **Primer carácter de cada palabra.** Crear una lista por comprensión que contenga el primer carácter de cada palabra.
2. **Longitud de cada palabra.** Crear una lista por comprensión que contenga la longitud de cada palabra.
3. **Cantidad total de caracteres (Reduce).** Utilizando Reduce, calcular la cantidad total de caracteres que contienen todas las palabras de la lista lst.

Actividad práctica.

1. **Comprensión · Crear lista.** Utilizando las rutinas de random (import random), crear una lista por comprensión. La lista debe contener 50 números enteros al azar, en el rango de 1 a 100.
2. **Filter · Filtrar elementos.** Utilizando la función Filter, separar los números generados en dos colecciones: una lista con los mayores a 50 y otra lista con los menores o iguales a 50.
3. **Map · Transformar datos.** Utilizando la función Map, procesar la lista inicial: obtener una nueva lista que contenga solo los números impares pero con su signo invertido (negativos).
4. **Reduce · Reducir a un valor.** Utilizando la función Reduce, realizar la consolidación: obtener la sumatoria total de la lista que fue generada en el paso anterior.

Qué practica

Combinar las cuatro herramientas. Para “solo los impares, negativos” se encadenan: `filter` elige los impares y `map` les cambia el signo.

Para tener en cuenta

- Con `random.seed(2026)` los números al azar salen siempre iguales, para poder comparar con esta salida. Sacando esa línea, cambian en cada ejecución.
- El archivo de la cátedra resolvía una versión anterior del paso de `map` (dejaba los pares positivos y los impares negativos); acá se sigue el enunciado 2026.

ejercicio.py

```python
import random
from functools import reduce

random.seed(2026)        # así la salida es siempre la misma (sacalo para que cambie)

lst = ["Aruba", "Jamaica", "Bermuda", "Bahama", "Key Largo", "Montigo"]

# 01. Primer carácter de cada palabra (comprensión)
iniciales = [p[0] for p in lst]
print("01.", iniciales)

# 02. Longitud de cada palabra (comprensión)
longitudes = [len(p) for p in lst]
print("02.", longitudes)

# 03. Cantidad total de caracteres (reduce)
print("03.", reduce(lambda a, b: a + b, longitudes))

# Actividad: 50 enteros al azar entre 1 y 100 (comprensión)
numeros = [random.randint(1, 100) for _ in range(50)]
print("Números:", numeros)

# Filter: mayores a 50 y menores o iguales a 50
mayores = list(filter(lambda x: x > 50, numeros))
menores = list(filter(lambda x: x <= 50, numeros))
print("Mayores a 50:", mayores)
print("Menores o iguales a 50:", menores)

# Map: solo los impares, con el signo invertido (filter elige los impares, map los niega)
impares_negativos = list(map(lambda x: -x, filter(lambda x: x % 2 != 0, numeros)))
print("Impares con signo invertido:", impares_negativos)

# Reduce: sumatoria de la lista anterior
print("Sumatoria:", reduce(lambda a, b: a + b, impares_negativos))
```

Salida al correrlo

```
01. ['A', 'J', 'B', 'B', 'K', 'M']
02. [5, 7, 7, 6, 9, 7]
03. 41
Números: [16, 41, 65, 66, 83, 14, 29, 77, 80, 72, 54, 74, 71, 94, 100, 99, 63, 97, 99, 76, 57,
31, 1, 79, 11, 15, 37, 13, 58, 2, 88, 63, 87, 41, 27, 51, 33, 45, 46, 49, 96, 66, 82, 10, 93, 44,
12, 72, 69, 38]
Mayores a 50: [65, 66, 83, 77, 80, 72, 54, 74, 71, 94, 100, 99, 63, 97, 99, 76, 57, 79, 58, 88,
63, 87, 51, 96, 66, 82, 93, 72, 69]
Menores o iguales a 50: [16, 41, 14, 29, 31, 1, 11, 15, 37, 13, 2, 41, 27, 33, 45, 46, 49, 10,
44, 12, 38]
Impares con signo invertido: [-41, -65, -83, -29, -77, -71, -99, -63, -97, -99, -57, -31, -1,
-79, -11, -15, -37, -13, -63, -87, -41, -27, -51, -33, -45, -49, -93, -69]
Sumatoria: -1526
```